

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ  
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ  
імені ІГОРЯ СІКОРСЬКОГО”  
НАВЧАЛЬНО-НАУКОВИХ ІНСТИТУТ АТОМНОЇ ТА ТЕПЛОВОЇ  
ЕНЕРГЕТИКИ  
КАФЕДРА ЦИФРОВИХ ТЕХНОЛОГІЙ В ЕНЕРГЕТИЦІ

РОЗРАХУНКОВО - ГРАФІЧНА РОБОТА  
з дисципліни " Візуалізація графічної та  
геометричної інформації"

ВАРІАНТ - 17

Виконав студент 5-го курсу  
групи ТР-51мп  
Козурак Трифон

Київ 2025

# **1. Опис завдання**

## **1.1. Мета роботи**

Метою даної роботи є розробка інтерактивного WebGL-додатку для візуалізації тривимірної параметричної поверхні з реалізацією складної моделі освітлення Фонга, нормального мапування та текстуровання.

## **1.2. Постановка задачі**

У ході роботи необхідно вирішити наступні задачі:

- Реалізувати математичну модель параметричної поверхні на основі заданих рівнянь
- Створити механізм динамічної генерації геометрії поверхні
- Реалізувати модель освітлення Фонга з підтримкою текстур
- Інтегрувати техніку нормального мапування для покращення деталізації
- Забезпечити інтерактивне керування камерою та параметрами сцени

## **1.3. Очікуваний результат**

В результаті має бути створений веб-додаток, що дозволяє взаємодіяти з 3D поверхнею, змінювати її параметри, регулювати освітлення та текстури в реальному часі.

## 2. Теоретичні відомості

### 2.1. Параметричні поверхні

Параметрична поверхня визначається векторною функцією двох параметрів  $u$  та  $v$ :

$$S(u,v) = [X(u,v), Y(u,v), Z(u,v)]$$

У даній роботі використовується поверхня, задана рівняннями:

$$x = (R \cdot \cos(v) + a \cdot (1 - \sin(v)) \cdot \cos(n \cdot u)) \cdot \cos(u)$$

$$y = (R \cdot \cos(v) + a \cdot (1 - \sin(v)) \cdot \cos(n \cdot u)) \cdot \sin(u)$$

$$z = R \cdot \sin(v)$$

де:

- $R$  - основний радіус поверхні
- $a$  - амплітуда модуляцій
- $n$  - кількість "пелюсток" поверхні
- $u \in [0, 2\pi]$  - параметр по колу
- $v \in [0, \pi/2]$  - параметр по висоті

### 2.2. Модель освітлення Фонга

Модель Фонга включає три компоненти освітлення:

**Амбієнтне освітлення:**

$$I_{ambient} = k_a \times I_a$$

**Дифузне освітлення (закон Ламберта):**

$$I_{diffuse} = k_d \times I_l \times (N \cdot L)$$

**Спекулярне освітлення:**

$$I_{specular} = k_s \times I_l \times (R \cdot V)^{shininess}$$

**Загальна інтенсивність:**

$$I_{total} = I_{ambient} + I_{diffuse} + I_{specular}$$

### 2.3. Нормальне мапування

Нормальне мапування - техніка, що дозволяє імітувати складну геометрію поверхні за допомогою спеціальної текстури, що містить інформацію про нормалі. Для коректного застосування використовується TVN-матриця (Tangent-Bitangent-Normal):

$$TBN = [T, B, N]$$

де:

- ***T** - вектор дотичної*
- ***B** - вектор бідотичної*
- ***N** - вектор нормалі*

Перетворення нормалі з текстури в простір світу:

$$N_{world} = TBN \times N_{texture}$$

## 2.4. WebGL та шейдери

WebGL надає низькорівневий API для рендерингу 3D графіки в браузері.

Основними компонентами є:

- **Vertex Shader** - обробляє вершини, виконує просторові перетворення
- **Fragment Shader** - обчислює кольори пікселів, реалізує освітлення
- **Buffer Objects** - зберігають дані геометрії на GPU
- **Textures** - зображення для покращення візуальної якості

## 3. Деталі реалізації

### 3.1. Архітектура програми

Програма реалізована за модульним принципом з наступними основними компонентами:

- **index.html** - головний HTML-документ з інтерфейсом користувача
- **main.js** - основний файл ініціалізації та керування програмою
- **Model.js** - клас для роботи з 3D моделями та геометрією
- **shader.gpu** - шейдерні програми (вершинний та фрагментний)
- **Utils/** - допоміжні бібліотеки (матриці, керування камерою)

### 3.2. Генерація геометрії поверхні

Функція CreateSurfaceData() генерує вершини, нормалі, текстурні координати

```
function CreateSurfaceData(n = 6, uSteps = 50, vSteps = 50) {
    let vertices = [];
    let normals = [];
    let texCoords = [];
    let tangents = [];

    for (let i = 0; i <= vSteps; i++) {
        let v = (i / vSteps) * Math.PI / 2;
        for (let j = 0; j <= uSteps; j++) {
            let u = (j / uSteps) * 2 * Math.PI;

            // Обчислення позиції вершини
            let x = (R * Math.cos(v) + a * (1 - Math.sin(v)) * Math.cos(n * u)) *
Math.cos(u);
            // ... аналогічно для y, z

            // Обчислення нормалей через векторний добуток
            let dx_du = ... // похідна по u
            let dy_du = ...
            let normal_x = dy_du * dz_dv - dz_du * dy_dv;
            // ...

        }
    }
}
```

### 3.3. Система освітлення

Реалізовано два режими освітлення:

- **Статичне** - джерело світла зафіксоване в просторі

- **Динамічне** - джерело світла рухається по колу

```
// У головному циклі рендерингу
if (isLightStatic) {
    lightPos[0] = 8;
    lightPos[1] = 4;
    lightPos[2] = 0;
} else {
    let t = performance.now() * 0.001;
    lightPos[0] = lightRadius * Math.cos(t);
    lightPos[2] = lightRadius * Math.sin(t);
}
```

### 3.4. Робота з текстурами

Програма використовує три типи текстур:

- **Diffuse texture** - основний колір поверхні
- **Specular texture** - карта блискучості
- **Normal texture** - карта нормалей для деталізації

Завантаження текстур реалізовано асинхронно:

```
async function loadTextures() {
    diffuseTexture = await loadTexture('Utils/textures/diffuse.jpg');
    specularTexture = await loadTexture('Utils/textures/specular.jpg');
    normalTexture = await loadTexture('Utils/textures/normal.jpg');
}
```

### 3.5. Інтерактивні елементи

- **Trackball контролер** - обертання камери перетягуванням миші
- **Масштабування** - колесом миші
- **Клавіатурне керування** - регулювання текстурних параметрів
- **Радіокнопки** - перемикання режимів освітлення

## 4. Інструкція користувача

### 4.1. Запуск програми

Для коректної роботи програми необхідний сучасний браузер з підтримкою WebGL (Chrome, Firefox, Safari, Edge). Просто відкрийте файл index.html у браузері (Рис.1)

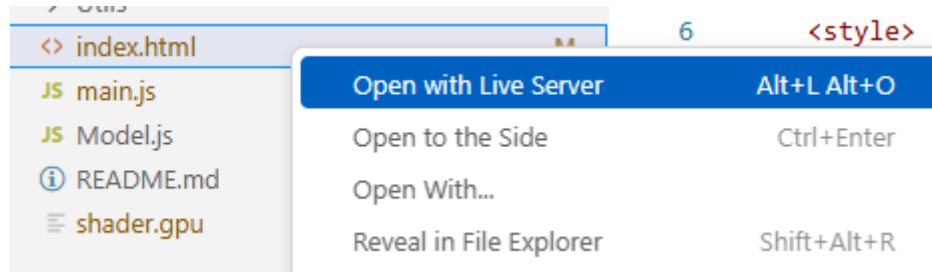


Рис.1

### 4.2. Основні елементи керування

#### Панель параметрів поверхні

- **Параметр n** - визначає кількість "пелюсток" поверхні (1-50)
- **uSteps** - кількість кроків по параметру u (20-200)
- **vSteps** - кількість кроків по параметру v (20-200)
- **Кнопка "Update Surface"** - застосовує змінені параметри

#### Режими освітлення

- **Static** - статичне джерело світла
- **Dynamic** - джерело світла рухається по колу

#### Клавіатурне керування текстурами

- **W/S** - переміщення точки по параметру V
- **A/D** - переміщення точки по параметру U
- **Q/E** - масштабування текстури по U
- **Z/X** - масштабування текстури по V

### 4.3. Керування камерою

- **Перетягування мишею** - обертання камери навколо об'єкта
- **Колесо миші** - наближення/віддалення камери

### 4.4. Приклад роботи програми

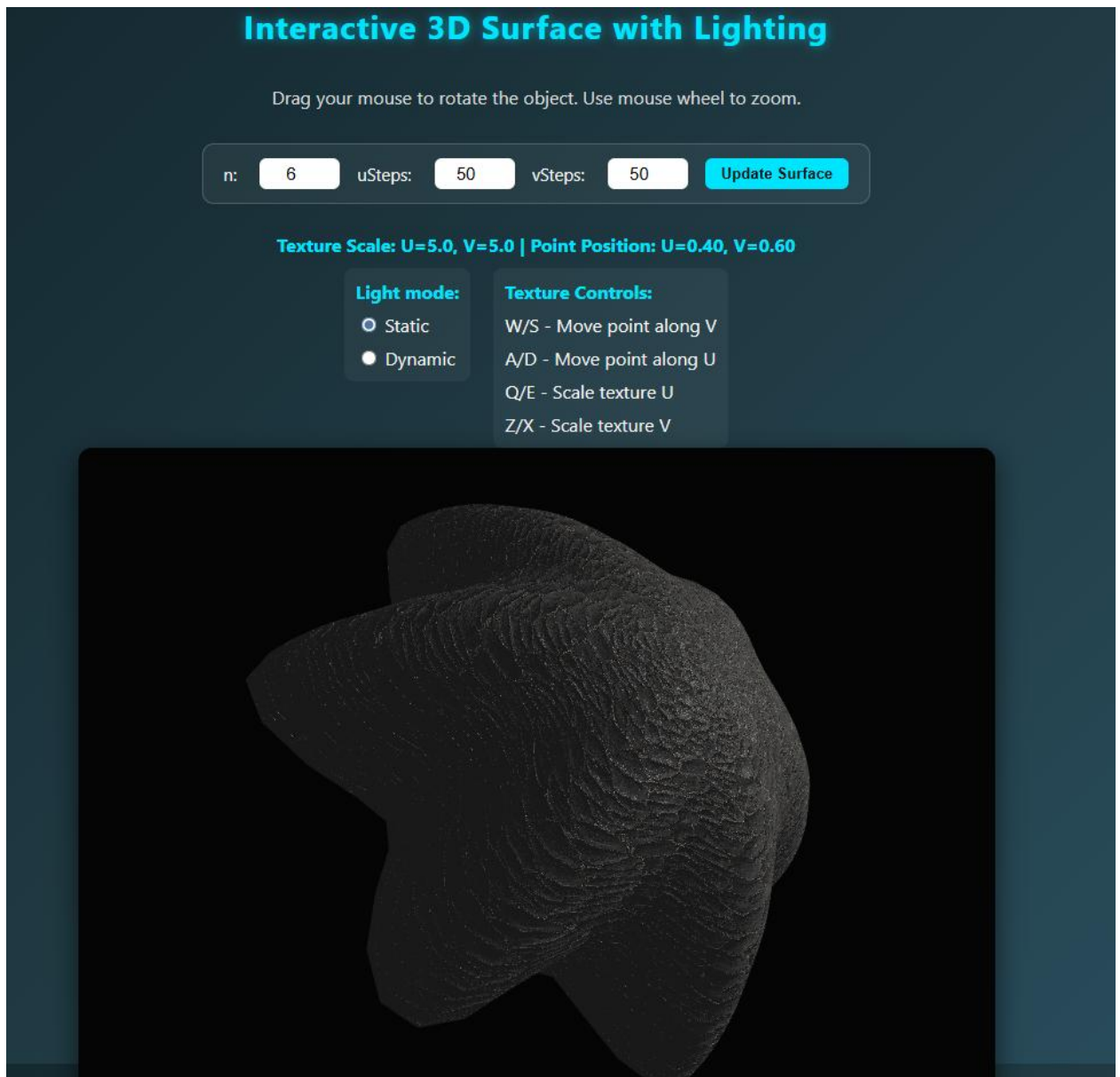


Рис.2 – Результат роботи програми

На зображенні показано типове представлення програми:

1. **3D поверхня** з застосованими текстурами та освітленням
2. **Джерело світла** у вигляді білої сфери
3. **Інтерфейс керування** з усіма доступними параметрами



## 5. Додатки

### 5.1. Ключові фрагменти реалізації

Головний цикл рендерингу (main.js)

```
function draw() {  
    // Оновлення розмірів canvas  
    const resized = resizeCanvasToDisplaySize(canvasGlobal);  
    if (resized) {  
        gl.viewport(0, 0, canvasGlobal.width, canvasGlobal.height);  
        projectionMatrix = m4.perspective(Math.PI/6,  
canvasGlobal.width/canvasGlobal.height, 0.1, 100);  
    }  
  
    gl.clearColor(0.02, 0.02, 0.02, 1);  
    gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);  
  
    // Керування джерелом світла  
    const lightModeValue =  
document.querySelector('input[name="lightMode"]:checked').value;  
    const isLightStatic = (lightModeValue === "static");  
  
    if (isLightStatic) {  
        lightPos[0] = 8; lightPos[1] = 4; lightPos[2] = 0;  
    } else {  
        let t = performance.now() * 0.001;  
        let lightRadius = 8;  
        lightPos[0] = lightRadius * Math.cos(t);  
        lightPos[1] = 4;  
        lightPos[2] = lightRadius * Math.sin(t);  
    }  
  
    // Оновлення матриць  
    viewMatrix = m4.translation(0, 0, zoom);  
    modelMatrix = spaceball.getViewMatrix();  
    normalMatrix4 = m4.transpose(m4.inverse(modelMatrix));  
  
    // Передача параметрів у шейдери  
    shProgram.Use();  
    gl.uniformMatrix4fv(shProgram.iModelMatrix, false, modelMatrix);  
    gl.uniformMatrix4fv(shProgram.iViewMatrix, false, viewMatrix);  
    gl.uniformMatrix4fv(shProgram.iProjectionMatrix, false, projectionMatrix);  
    gl.uniformMatrix3fv(shProgram.iNormalMatrix, false, normalMatrix);  
    gl.uniform3fv(shProgram.uLightPosition, lightPos);  
  
    // Відображення поверхні  
    surface.Draw();  
  
    // Відображення джерела світла  
    lightModelMatrix = m4.translation(lightPos[0], lightPos[1], lightPos[2]);
```

```

gl.uniformMatrix4fv(shProgram.iModelMatrix, false, lightModelMatrix);
gl.uniform1i(shProgram.uIsLight, 1);
lightSphere.Draw();

requestAnimationFrame(draw);
}

```

## Генерація геометрії поверхні (Model.js)

```

function CreateSurfaceData(n = 6, uSteps = 50, vSteps = 50) {
    let vertices = [], normals = [], texCoords = [], tangents = [];
    let R = 1, a = 0.24;

    for (let i = 0; i <= vSteps; i++) {
        let v = (i / vSteps) * Math.PI / 2;
        for (let j = 0; j <= uSteps; j++) {
            let u = (j / uSteps) * 2 * Math.PI;

            // Обчислення позиції вершини
            let x = (R * Math.cos(v) + a * (1 - Math.sin(v)) * Math.cos(n * u)) *
Math.cos(u);
            let y = (R * Math.cos(v) + a * (1 - Math.sin(v)) * Math.cos(n * u)) *
Math.sin(u);
            let z = R * Math.sin(v);

            vertices.push(x, y, z);
            texCoords.push(j / uSteps, i / vSteps);

            // Обчислення похідних для нормалей
            let dx_du = -(R * Math.cos(v) + a * (1 - Math.sin(v)) * Math.cos(n *
u)) * Math.sin(u)
                - a * (1 - Math.sin(v)) * n * Math.sin(n * u) *
Math.cos(u);
            let dy_du = (R * Math.cos(v) + a * (1 - Math.sin(v)) * Math.cos(n *
u)) * Math.cos(u)
                - a * (1 - Math.sin(v)) * n * Math.sin(n * u) *
Math.sin(u);
            let dz_du = 0;

            let dx_dv = (-R * Math.sin(v) - a * Math.cos(v) * Math.cos(n * u)) *
Math.cos(u);
            let dy_dv = (-R * Math.sin(v) - a * Math.cos(v) * Math.cos(n * u)) *
Math.sin(u);
            let dz_dv = R * Math.cos(v);

            // Векторний добуток для нормалі
            let normal_x = dy_du * dz_dv - dz_du * dy_dv;
            let normal_y = dz_du * dx_dv - dx_du * dz_dv;
            let normal_z = dx_du * dy_dv - dy_du * dx_dv;

```

```

        // Нормалізація
        let length = Math.sqrt(normal_x * normal_x + normal_y * normal_y +
normal_z * normal_z);
        if (length > 0) {
            normal_x /= length; normal_y /= length; normal_z /= length;
        }

        normals.push(normal_x, normal_y, normal_z);

        // Обчислення дотичного вектора
        let tangentLength = Math.sqrt(dx_du * dx_du + dy_du * dy_du + dz_du *
dz_du);
        if (tangentLength > 0) {
            tangents.push(dx_du/tangentLength, dy_du/tangentLength,
dz_du/tangentLength);
        } else {
            tangents.push(1, 0, 0);
        }
    }
}

return { vertices, normals, texCoords, tangents };
}

```

## 5.2. Шейдерні програми

### Вершинний шейдер (shader.gpu)

```

attribute vec3 vertex;
attribute vec3 normal;
attribute vec2 texCoord;
attribute vec3 tangent;

uniform mat4 ModelMatrix;
uniform mat4 ViewMatrix;
uniform mat4 ProjectionMatrix;
uniform mat3 NormalMatrix;
uniform vec2 uTextureScale;
uniform vec2 uPointPosition;

varying vec3 vNormal;
varying vec3 vWorldPos;
varying vec2 vTexCoord;
varying vec3 vTangent;
varying vec3 vBitangent;

void main() {
    vec4 worldPos = ModelMatrix * vec4(vertex, 1.0);
    vWorldPos = worldPos.xyz;
}

```

```

    // Ортогоналізація Грама-Шмідта для TBN матриці
    vec3 T = normalize(NormalMatrix * tangent);
    vec3 N = normalize(NormalMatrix * normal);
    T = normalize(T - dot(T, N) * N);
    vec3 B = cross(N, T);

    vNormal = N;
    vTangent = T;
    vBitangent = B;

    // Масштабування текстури відносно обраної точки
    vTexCoord = (texCoord - uPointPosition) * uTextureScale + uPointPosition;

    gl_Position = ProjectionMatrix * ViewMatrix * worldPos;
}

```

## Фрагментний шейдер з нормальним мапуванням

```

precision highp float;

varying vec3 vNormal;
varying vec3 vWorldPos;
varying vec2 vTexCoord;
varying vec3 vTangent;
varying vec3 vBitangent;

uniform vec3 uLightPosition;
uniform vec3 uViewPosition;
uniform vec3 uAmbientColor;
uniform float uShininess;
uniform bool uIsLight;

uniform sampler2D uDiffuseTexture;
uniform sampler2D uSpecularTexture;
uniform sampler2D uNormalTexture;

void main() {
    if (uIsLight) {
        gl_FragColor = vec4(1.0, 1.0, 1.0, 1.0);
    } else {
        // Нормальне мапування
        vec3 normalMap = texture2D(uNormalTexture, vTexCoord).rgb * 2.0 - 1.0;
        mat3 TBN = mat3(vTangent, vBitangent, vNormal);
        vec3 N = normalize(TBN * normalMap);

        vec3 L = normalize(uLightPosition - vWorldPos);
        vec3 V = normalize(uViewPosition - vWorldPos);
        vec3 R = reflect(-L, N);
    }
}

```

```

    // Модель освітлення Фонга
    vec3 ambient = uAmbientColor;

    vec3 diffuseColor = texture2D(uDiffuseTexture, vTexCoord).rgb;
    float diff = max(dot(N, L), 0.0);
    vec3 diffuse = diffuseColor * diff;

    float specularStrength = 1.0 - texture2D(uSpecularTexture, vTexCoord).r;
    float spec = 0.0;
    if (diff > 0.0) {
        spec = pow(max(dot(V, R), 0.0), uShininess);
    }
    vec3 specular = vec3(specularStrength) * spec;

    vec3 color = ambient + diffuse + specular;
    gl_FragColor = vec4(color, 1.0);
}
}

```

## Клас для роботи з 3D моделями

```

function Model(name) {
    this.name = name;
    this.iVertexBuffer = gl.createBuffer();
    this.iNormalBuffer = gl.createBuffer();
    this.iTexCoordBuffer = gl.createBuffer();
    this.iTangentBuffer = gl.createBuffer();
    this.iIndexBuffer = gl.createBuffer();

    this.BufferData = function(vertices, normals, texCoords, tangents, indices) {
        gl.bindBuffer(gl.ARRAY_BUFFER, this.iVertexBuffer);
        gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(vertices),
gl.STATIC_DRAW);

        gl.bindBuffer(gl.ARRAY_BUFFER, this.iNormalBuffer);
        gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(normals),
gl.STATIC_DRAW);

        gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, this.iIndexBuffer);
        gl.bufferData(gl.ELEMENT_ARRAY_BUFFER, new Uint16Array(indices),
gl.STATIC_DRAW);

        this.indexCount = indices.length;
    }

    this.Draw = function() {
        gl.bindBuffer(gl.ARRAY_BUFFER, this.iVertexBuffer);
        gl.vertexAttribPointer(shProgram.iAttribVertex, 3, gl.FLOAT, false, 0,
0);

        gl.enableVertexAttribArray(shProgram.iAttribVertex);
    }
}

```

```
gl.bindBuffer(gl.ARRAY_BUFFER, this.iNormalBuffer);
gl.vertexAttribPointer(shProgram.iAttribNormal, 3, gl.FLOAT, false, 0,
0);
gl.enableVertexAttribArray(shProgram.iAttribNormal);

gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, this.iIndexBuffer);
gl.drawElements(gl.TRIANGLES, this.indexCount, gl.UNSIGNED_SHORT, 0);
    }
}
```